



Django

Capítulo 6 Django ORM

Django

Capítulo 6 - Django ORM

Mapeamento objeto-relacional (Object-relational mapping - ORM) é uma técnica para aproximar o paradigma de desenvolvimento de aplicações orientadas a objetos do paradigma do banco de dados relacional (tabelas). Tem sido muito utilizada atualmente e pode trazer muitas vantagens para quem a implementa, tais como: fácil reutilização do código, manutenibilidade e legibilidade da estrutura da aplicação. Através do uso da Orientação a Objetos, conseguiremos criar e mapear um objeto para que ele se comporte da forma adequada em uma estrutura de banco de dados relacional. Criaremos nossas views e nelas implementamos uma série de métodos para trabalhar com as técnicas ORM.

Django

Capítulo 6 - Django ORM

O primeiro passo é capturar o que virá na url. Será utilizado o método GET do request(parâmetro do método view) para que as buscas possam ser compartilhadas via url. Adicione a leitura de parâmetros na classe view de produtos, loja/views/**ProdutoView.py**. Assim:

```
produto = request.GET.get("produto")
destaque = request.GET.get("destaque")
promocao = request.GET.get("promocao")
categoria = request.GET.get("categoria")
fabricante = request.GET.get("fabricante")
```

OBS: O nome dentro do parêntese indica como deve ser escrito o parâmetro na url do navegador para que a variável possa capturar o valor do navegador

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponse
2  def list_produto_view(request, id=None):
3      produto = request.GET.get("produto")
4      destaque = request.GET.get("destaque")
5      promocao = request.GET.get("promocao")
6      categoria = request.GET.get("categoria")
7      fabricante = request.GET.get("fabricante")
8      if id is None:
9          id = 0
10     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

Django

Capítulo 6 - Django ORM

Visualizarmos os dados consultados no console(nesse caso o cmd). Para realizar essa consulta e mostrar no console, inclua na view as linhas que realizam a consulta e mostram o resultado no console. É necessário importar a classe de produto para a view. Import:

```
from loja.models import Produto
```

Comando para mostrar resultado da consulta no cmd:

```
produtos = Produto.objects.all()  
print(produtos)
```

OBS: Do jeito que está, mostrará todos os produtos do banco no console, graças ao `Produto.objects.all()`

```
loja > views > ProdutoView.py > ...  
1  from django.http import HttpResponse  
2  #Adicione o import abaixo  
3  from loja.models import Produto  
4  def list_produto_view(request, id=None):  
5      produto = request.GET.get("produto")  
6      destaque = request.GET.get("destaque")  
7      promocao = request.GET.get("promocao")  
8      categoria = request.GET.get("categoria")  
9      fabricante = request.GET.get("fabricante")  
10     #Adicione as duas linhas abaixo  
11     produtos = Produto.objects.all()  
12     print(produtos)  
13     if id is None:  
14         id = 0  
15     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

Django

Capítulo 6 - Django ORM

Assim, ao realizar a chamada(rodar) à aplicação o sistema mostrará a listagem completa de produtos no console. Isso ocorre pois a consulta realizada ("Produto.objects.all()") ainda não considera os parâmetros, nem necessita.

```
(venv) C:\Repos\django\thiago>python manage.py runserver 127.0.0.1:8080
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
January 18, 2024 - 03:48:10
Django version 4.2.7, using settings 'lojaAdmin.settings.development'
Starting development server at http://127.0.0.1:8080/
Quit the server with CTRL-BREAK.

<QuerySet [<Produto: Windows 11>, <Produto: Monitor 24 polegadas>, <Produto: Teste de data>]>
[18/Jan/2024 03:48:13] "GET /produto/ HTTP/1.1" 200 39
```



Django

Capítulo 6 - Django ORM

Caso a consulta ("Produto.objects.all()") seja alterada para "Produto.objects.first()" será retornado apenas o primeiro objeto da tabela feita com base no model de Produto.

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponseRedirect
2  from loja.models import Produto
3  def list_produto_view(request, id=None):
4      produto = request.GET.get("produto")
5      destaque = request.GET.get("destaque")
6      promocao = request.GET.get("promocao")
7      categoria = request.GET.get("categoria")
8      fabricante = request.GET.get("fabricante")
9      produtos = Produto.objects.first()
10     print(produtos)
11     if id is None:
12         id = 0
13     return HttpResponseRedirect('<h1>Produto de id %s!</h1>' % id)
```

```
System check identified no issues (0 silenced).
January 18, 2024 - 03:49:39
Django version 4.2.7, using settings 'lojaAdmin.settings.development'
Starting development server at http://127.0.0.1:8080/
Quit the server with CTRL-BREAK.

Windows 11
[18/Jan/2024 03:49:58] "GET /produto/ HTTP/1.1" 200 39
```

Django

Capítulo 6 - Django ORM

Para visualizar os parâmetros informados na string de consulta no console inclua as seguintes linhas para visualizar os parâmetros informados antes do return e do "if id is None:" :

```
if destaque is not None:
    print(destaque)
if produto is not None:
    print(produto)
if promocao is not None:
    print(promocao)
if categoria is not None:
    print(categoria)
if fabricante is not None:
    print(fabricante)
```

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponse
2  from loja.models import Produto
3  def list_produto_view(request, id=None):
4      produto = request.GET.get("produto")
5      destaque = request.GET.get("destaque")
6      promocao = request.GET.get("promocao")
7      categoria = request.GET.get("categoria")
8      fabricante = request.GET.get("fabricante")
9      produtos = Produto.objects.first()
10     print(produtos)
11     if destaque is not None:
12         print(destaque)
13     if produto is not None:
14         print(produto)
15     if promocao is not None:
16         print(promocao)
17     if categoria is not None:
18         print(categoria)
19     if fabricante is not None:
20         print(fabricante)
21     if id is None:
22         id = 0
23     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

Django

Capítulo 6 - Django ORM

Faça a consulta para pesquisar os produtos passando o parâmetro de nome do produto com o link `http://127.0.0.1:8080/produto/?produto=Monitor`

no final do link temos `"?produto=Monitor"` a interrogação indica que vai ser passado parâmetro(s), "produto" tem que ser o nome de uma string colocado no parâmetro do `GET.get` do request, "Monitor" é o valor que atribuímos a esse parâmetro.



OBS: o Windows 11 está aparecendo porque não excluimos o print da variável produtos e Monitor é o exato valor que passamos no parâmetro da url do navegador.

```
Windows 11  
Monitor  
[18/Jan/2024 03:58:27] "GET /produto/?produto=Monitor HTTP/1.1" 200 39
```

Django

Capítulo 6 - Django ORM

Para adicionar filtros na consulta utilize o método `filter()` ao invés do `first()` e como parâmetro deve-se passar o campo que filtramos e o valor desse campo que desejamos que o objeto a ser procurado tenha. Adicione essa linha substituindo a linha do `first`:

```
produtos = Produto.objects.filter(Produto=produto)
```

OBS: Esse comando irá procurar objetos da classe `produto`, cujo campo `produto` seja igual ao valor da variável `produto` (`Produto = request.GET.get("produto")`)

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponse
2  from loja.models import Produto
3  def list_produto_view(request, id=None):
4      produto = request.GET.get("produto")
5      destaque = request.GET.get("destaque")
6      promocao = request.GET.get("promocao")
7      categoria = request.GET.get("categoria")
8      fabricante = request.GET.get("fabricante")
9      produtos = Produto.objects.filter(Produto=produto)
10     print(produtos)
11     if destaque is not None:
12         print(destaque)
13     if produto is not None:
14         print(produto)
15     if promocao is not None:
16         print(promocao)
17     if categoria is not None:
18         print(categoria)
19     if fabricante is not None:
20         print(fabricante)
21     if id is None:
22         id = 0
23     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

Django

Capítulo 6 - Django ORM

Para observar o resultado consulte Monitor 24 polegadas usando o link, ou inclua um produto que você tenha inserido no banco de dados. link :

[http://127.0.0.1:8080/produto/?produto=Monitor 24 polegadas](http://127.0.0.1:8080/produto/?produto=Monitor%2024%20polegadas)



```
#<QuerySet [<Produto: Monitor 24 polegadas>]>
#Monitor 24 polegadas
[18/Jan/2024 04:11:18] "GET /produto/?produto=Monitor%2024%20polegadas HTTP/1.1" 200 39
```

Django

Capítulo 6 - Django ORM

Agora vamos fazer com que filtre quaisquer Produtos que tenham algo igual a algum valor do(s) parâmetro(s) passado na url do navegador. Para isso encadeie os comando de condicional (**if**) desta forma e altere a consulta:

Parte 1:

```
produtos = Produto.objects.all()
if produto is not None:
    produtos = produtos.filter(Produto=produto)
if promocao is not None:
    produtos = produtos.filter(promocao=promocao)
if destaque is not None:
    produtos = produtos.filter(destaque=destaque)
```

Django

Capítulo 6 - Django ORM

Parte 2:

if categoria is not None:

 produtos = produtos.filter(categoria=categoria)

if fabricante is not None:

 produtos = produtos.filter(fabricante=fabricante)

if id is not None:

 produtos = produtos.filter(id=id)

print(produtos)

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponse
2  from loja.models import Produto
3  def list_produto_view(request, id=None):
4      produto = request.GET.get("produto")
5      destaque = request.GET.get("destaque")
6      promocao = request.GET.get("promocao")
7      categoria = request.GET.get("categoria")
8      fabricante = request.GET.get("fabricante")
9      produtos = Produto.objects.all()
10     #print(produtos)
11     if produto is not None:
12         produtos = produtos.filter(Produto=produto)
13     if promocao is not None:
14         produtos = produtos.filter(promocao=promocao)
15     if destaque is not None:
16         produtos = produtos.filter(destaque=destaque)
17     if categoria is not None:
18         produtos = produtos.filter(categoria=categoria)
19     if fabricante is not None:
20         produtos = produtos.filter(fabricante=fabricante)
21     if id is not None:
22         produtos = produtos.filter(id=id)
23     print(produtos)
24     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

OBS: Neste código, primeiro pegamos todos os produtos do banco, depois filtramos objetos que tenham algum valor igual a um dos GET atribuído, ou seja, diferente de None. Depois atribuímos no console o objeto achado.

Django

Capítulo 6 - Django ORM

Antes de rodar vamos colocar o valor de promoção do Windows 11 como False e colocar o destaque do Monitor 24 polegadas como True no painel do Admin para podermos ver mais variações de resultados no console. Use esses links:

<http://127.0.0.1:8080/produto/?promocao=True>

<http://127.0.0.1:8080/produto/?destaque=True>

<http://127.0.0.1:8080/produto/?categoria=1>

<http://127.0.0.1:8080/produto/?categoria=2>

<http://127.0.0.1:8080/produto/?fabricante=1>

<http://127.0.0.1:8080/produto/?fabricante=2>

<http://127.0.0.1:8080/produto/1>

<http://127.0.0.1:8080/produto/2>

Django

Capítulo 6 - Django ORM

Para consultar pelo nome dos conteúdos dos campos de chave estrangeira, ou seja, fabricante e categoria, faça o seguinte ajuste no comando de filtragem:

```
if categoria is not None:
```

```
    produtos = produtos.filter(categoria__Categoria=categoria)
```

```
if fabricante is not None:
```

```
    produtos = produtos.filter(fabricante__Fabricante=fabricante)
```

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponseRedirect
2  from loja.models import Produto
3  def list_produto_view(request, id=None):
4      produto = request.GET.get("produto")
5      destaque = request.GET.get("destaque")
6      promocao = request.GET.get("promocao")
7      categoria = request.GET.get("categoria")
8      fabricante = request.GET.get("fabricante")
9      produtos = Produto.objects.all()
10     #print(produtos)
11     if produto is not None:
12         produtos = produtos.filter(Produto=produto)
13     if promocao is not None:
14         produtos = produtos.filter(promocao=promocao)
15     if destaque is not None:
16         produtos = produtos.filter(destaque=destaque)
17     if categoria is not None:
18         produtos = produtos.filter(categoria__Categoria=categoria)
19     if fabricante is not None:
20         produtos = produtos.filter(fabricante__Fabricante=fabricante)
21     if id is not None:
22         produtos = produtos.filter(id=id)
23     print(produtos)
24     return HttpResponseRedirect('<h1>Produto de id %s!</h1>' % id)
```

OBS: quando ele está só com o nome da classe ele entendia que era para procurar o id da classe, porém, com o sufixo `__Categoria` e `__Fabricante`, ele passa a procurar com base no nome

Django

Capítulo 6 - Django ORM

Links para testar:

<http://127.0.0.1:8080/produto/?categoria=Hardware>

<http://127.0.0.1:8080/produto/?categoria=Software>

← ↻ ⓘ 127.0.0.1:8080/produto/?categoria=Software

Listagem de 1 ou mais produtos

```
<QuerySet [<Produto: Windows 11>]>  
[18/Jan/2024 04:36:26] "GET /produto/?categoria=Software HTTP/1.1" 200 39
```

← ↻ ⓘ 127.0.0.1:8080/produto/?categoria=Hardware

Listagem de 1 ou mais produtos

```
<QuerySet [<Produto: Monitor 24 polegadas>, <Produto: Teste de data>]>  
[18/Jan/2024 04:37:26] "GET /produto/?categoria=Hardware HTTP/1.1" 200 39
```

Django

Capítulo 6 - Django ORM

Para consultar strings parciais, ou seja, não precisa escrever tudo igual a string original de forma similar ao que é feito com o comando like da linguagem SQL, ou seja, fazendo uma busca pelo conteúdo ao invés de filtrar apenas as string exatamente iguais, utilize o sufixo `__contains` no campo de filtragem. Faça a alteração na consulta do nome do produto, colocando dentro do filter, após o campo, o sufixo `__contains`. Assim:

```
produtos = produtos.filter(Produto__contains=produto )
```

← ↻ ⓘ 127.0.0.1:8080/produto/?produto=Monitor

```
<QuerySet [<Produto: Monitor 24 polegadas>]>  
[18/Jan/2024 04:41:13] "GET /produto/?produto=Monitor HTTP/1.1" 200 39
```

ⓘ 127.0.0.1:8080/produto/?produto=Win

```
<QuerySet [<Produto: Windows 11>]>  
[18/Jan/2024 04:42:04] "GET /produto/?produto=Win HTTP/1.1" 200 39
```

Django

Capítulo 6 - Django ORM

Outra possibilidade de consulta é fazer comparações com campos de data ou número procurando por valores maiores, menores e outras situações. Nesses casos existem sufixos para cada caso, entre eles os abaixo listados:

__in: Cláusula IN(1,2,3). Exemplo: `.filter(id__in=[1,2,3]);`

__lt: Cláusula menor que (<). Exemplo: `.filter(user__date_joined__lt=datetime.now());`

__lte: Cláusula menor ou igual (<=). Exemplo:

`.filter(user__date_joined__lte=datetime.now());`

__gt: Cláusula maior que (>). Exemplo: `.filter(user__date_joined__gt=datetime.now());`

__gte: Cláusula maior ou igual (>=). Exemplo:

`.filter(user__date_joined__gte=datetime.now());`

Django

Capítulo 6 - Django ORM

Para consultar os produtos cadastrados nos últimos n dias faça as seguintes alterações no código, acrescente as bibliotecas de tratamento de data, acrescente um parâmetro que indique a quantidade de dias e, inclua o comando de filtragem com o sufixo desejado. Exemplo:

Primeiro inclua os seguintes imports:

```
from datetime import timedelta, datetime  
from django.utils import timezone
```

Depois inclua mais um GET no método list_produto_view para os dias:

```
dias = request.GET.get("dias")
```

Django

Capítulo 6 - Django ORM

Depois coloque a lógica do filtro para consultar os produtos cadastrados nos últimos dias, abaixo do “`produtos=Produto.objects.all()`”:

```
if dias is not None:
```

```
    now = timezone.now()
```

```
    now = now - timedelta(days = int(dias))
```

```
    produtos = produtos.filter(criado_em__gte=now)
```

```
loja > views > ProdutoView.py > ...
1  from django.http import HttpResponse
2  from loja.models import Produto
3  #Adicione os import a seguir
4  from datetime import timedelta, datetime
5  from django.utils import timezone
6  def list_produto_view(request, id=None):
7      produto = request.GET.get("produto")
8      destaque = request.GET.get("destaque")
9      promocao = request.GET.get("promocao")
10     categoria = request.GET.get("categoria")
11     fabricante = request.GET.get("fabricante")
12     #Adicione o GET a seguir
13     dias = request.GET.get("dias")
14     produtos = Produto.objects.all()
15     #Adicione o if abaixo
16     if dias is not None:
17         now = timezone.now()
18         now = now - timedelta(days = int(dias))
19         produtos = produtos.filter(criado_em__gte=now)
20     if produto is not None:
21         produtos = produtos.filter(Produto=produto)
22     if promocao is not None:
23         produtos = produtos.filter(promocao=promocao)
24     if destaque is not None:
25         produtos = produtos.filter(destaque=destaque)
26     if categoria is not None:
27         produtos = produtos.filter(categoria__Categoria=categoria)
28     if fabricante is not None:
29         produtos = produtos.filter(fabricante__Fabricante=fabricante)
30     if id is not None:
31         produtos = produtos.filter(id=id)
32     print(produtos)
33     return HttpResponse('<h1>Produto de id %s!</h1>' % id)
```

Django

Capítulo 6 - Django ORM

Testando a funcionalidade adicionada com os links:

Criados há menos 1 dia:

<http://127.0.0.1:8080/produto/?dias=1>

i 127.0.0.1:8080/produto/?dias=1

```
<QuerySet []>  
[18/Jan/2024 04:52:05] "GET /produto/?dias=1 HTTP/1.1" 200 39
```

Criados há menos 5 dia:

<http://127.0.0.1:8080/produto/?dias=5>

i 127.0.0.1:8080/produto/?dias=5

```
<QuerySet [<Produto: Windows 11>, <Produto: Monitor 24 polegadas>, <Produto: Teste de data>]>  
[18/Jan/2024 04:53:37] "GET /produto/?dias=5 HTTP/1.1" 200 39
```

Criados há menos 25 dia:

<http://127.0.0.1:8080/produto/?dias=25>

i 127.0.0.1:8080/produto/?dias=25

```
<QuerySet [<Produto: Windows 11>, <Produto: Monitor 24 polegadas>, <Produto: Teste de data>]>  
[18/Jan/2024 04:54:52] "GET /produto/?dias=25 HTTP/1.1" 200 39
```